

DocRot Detector

Team Name: Team 2

Team Members:

Marie Liske
Suchi Jain
Aaron Santhosh
Samuel Say
Portia Hamid
Anusua Halder

Supervisor:

Professor Sridhar Alagar

Coordinator:

Adarsh Gella

**Erik Jonsson School of Engineering and Computer Science University
of Texas at Dallas**

Date: May 2026

Table of Contents

Executive Summary

1. Introduction
2. Project Specification & Management
3. System Architecture & High-Level Design
4. Implementation & AI Orchestration
5. System Performance & Validation
6. Lessons Learned
7. Conclusion & Future Work References Acknowledgements

Executive Summary

DocRot Detector is a behavior-aware documentation rot detection platform developed by Team 2 as part of the CS 4485 Senior Capstone at the University of Texas at Dallas. The tool addresses a pervasive and underserved problem in software engineering: as codebases evolve, documentation becomes stale. Function signatures change, logic is restructured, and side effects shift, but docstrings and README files are rarely updated to match. DocRot Detector automates the detection of this drift.

Unlike simple text-diff tools, DocRot operates on semantic behavior. It parses Python source files into multi-dimensional fingerprints using AST analysis, tracks changes across control flow, side effects, API signatures, exception handling, and return patterns, maps those changes to linked documentation files, and generates scored alerts when documentation likely needs review. The system runs automatically via GitHub Actions on every push, persists results to Firebase Firestore via a Google Cloud Function, and surfaces findings through a live React dashboard at <https://docrot-detector.web.app/>. An optional AI layer powered by Groq (Llama 3.3 70B) generates specific, actionable documentation fix suggestions for flagged files and delivers them directly into GitHub Issues and the dashboard.

Over twelve weeks, the team delivered a fully deployed end-to-end system including a complete React frontend dashboard, a secure CI/CD pipeline using OIDC and Workload Identity Federation, an npm setup wizard for zero-friction onboarding, an auto-fix feature that opens pull requests to resolve flagged issues, incremental scanning using git diff, and parallelized fingerprinting and LLM calls for performance at scale.

1. Introduction

1.1 Motivation & Problem Statement

Software documentation is one of the most valuable and most neglected artifacts in a codebase. Developers write docstrings and README files when code is first created, but as the codebase evolves through refactors, feature additions, and bug fixes, the documentation rarely keeps pace. The result is documentation rot: a state where the documented behavior of a function or system no longer matches its actual behavior.

DocRot Detector was built to fill this gap. The motivation was to create something a real engineering team could install in minutes and benefit from immediately — a tool that runs silently on every push, flags only meaningful drift, and tells developers exactly what needs to be updated and why.

1.2 Objectives, Scope, and Goals Achieved

Objectives:

- Detect semantically meaningful code changes in Python files using AST-based fingerprinting
- Score changes by behavioral impact across multiple dimensions including control flow, side effects, API signatures, exception handling, and return behavior
- Map changed functions to linked documentation files via configurable doc_mappings in .docrot-config.json
- Generate human-readable and machine-readable rot reports on every scan
- Persist scan results, flags, and AI suggestions to a cloud database for historical tracking
- Surface findings through a live web dashboard with GitHub authentication
- Provide optional AI-generated documentation fix suggestions via an LLM integration
- Support an auto-fix workflow that opens pull requests to resolve flagged issues
- Automate the full pipeline via GitHub Actions with secure, keyless cloud authentication

Scope: The MVP targets Python repositories hosted on GitHub. All analysis is static and AST-based with no runtime execution required. The system triggers on push events via GitHub Actions. Out of scope for this release are non-Python languages, IDE plugin integrations, and webhook-based triggering.

Goals Achieved:

- Fully deployed AST-based semantic fingerprinting and documentation rot detection pipeline
- Configurable flagging thresholds via .docrot-config.json with per-project sensitivity tuning
- Report generation producing both .docrot-report.txt and .docrot-report.json on every scan
- Firebase Firestore backend with structured storage for scans, flags, fingerprint baselines, and AI suggestions
- GitHub Actions CI/CD pipeline using OIDC and Workload Identity Federation with no long-lived secrets
- React/TypeScript/Vite frontend dashboard live at <https://docrot-detector.web.app/>
- Groq-powered AI documentation fix suggestions integrated into GitHub Issues and the dashboard
- Auto-fix feature that generates documentation patches and opens pull requests against target repositories
- npm setup wizard enabling onboarding via a single npx command
- Incremental scanning using git diff to restrict analysis to changed files only
- Parallelized file fingerprinting and LLM suggestion calls for improved performance at scale

1.3 Proposed Solution

DocRot Detector operates as a three-layer system.

The analysis layer runs as a GitHub Action on every push. It walks the repository, parses Python files into semantic fingerprints per function and method, compares current fingerprints against a stored baseline, scores behavioral changes across seven dimensions, maps changed functions to linked documentation files using `.docrot-config.json`, and emits structured report artifacts. On the first run, the system creates a baseline and exits clean. On subsequent runs, it compares against that baseline to detect drift.

The persistence layer consists of a Google Cloud Function (`ingestScan`) that receives the scan payload via an OIDC-authenticated POST request and writes scan metadata, flags, AI suggestions, and fingerprint baselines into Firebase Firestore. This architecture ensures the GitHub Action never requires direct database write access, eliminating a significant class of credential exposure risk.

The presentation layer is a React single-page application that authenticates users via Firebase GitHub OAuth and reads from Firestore in real time. Users see a dashboard of repository health, per-repo rot scores, filterable issue tables with detail panels, scan history, and AI-generated fix suggestions. An auto-fix button triggers the Cloud Function to generate a documentation patch and open a pull request against the target repository.

1.4 Key Highlights

Technologies and breakthroughs:

- Behavior-aware semantic fingerprinting across seven dimensions (signature, control flow, conditions, calls, side effects, exception behavior, return behavior) makes the detector resilient to cosmetic changes while remaining sensitive to real behavioral drift
- Workload Identity Federation eliminates long-lived GCP credentials from the pipeline entirely, using short-lived OIDC tokens minted by GitHub Actions
- Groq/Llama 3.3 70B AI integration generates specific, targeted documentation edits for each flagged file, delivered into GitHub Issues and the Firestore-backed dashboard, with the API key securely managed server-side via Google Cloud Secret Manager
- Auto-fix pipeline closes the loop from detection to resolution by generating a documentation patch and opening a pull request against the target repository
- npm setup wizard enables any repository to onboard DocRot in under a minute with a single `npx` command
- Incremental scanning with git diff and parallelized `ThreadPoolExecutor` fingerprinting reduced scan time substantially for larger repositories

Key findings:

- Semantic fingerprinting significantly outperforms text-diff approaches for detecting meaningful documentation drift while ignoring cosmetic edits

- Routing all database writes through an authenticated Cloud Function is both more secure and more maintainable than granting the CI runner direct database access
- Configurable thresholds are essential to usability — without them, minor refactors generate alert fatigue that causes developers to ignore the tool

2. Project Specification & Management

2.1 The Project Specification (The AI's Blueprint + Link)

The team maintained a living specification document throughout the semester that served as the master instruction set for AI-assisted development sessions and the authoritative reference for all architectural decisions, module interfaces, and data schemas. The spec defined the full analysis pipeline, the `.docrot-config.json` configuration schema, the Firestore data model, the API contract between the GitHub Action and the Cloud Function, and the scoring model for change severity. It was version-controlled in the backend repository and updated via pull requests as the architecture evolved.

The spec was structured to be directly usable as context for AI coding sessions — each module section defined inputs, outputs, and interface contracts precisely enough that an AI agent given the relevant section could generate code that integrated without modification.

Backend repository (includes spec): https://github.com/SuchiiJain/CS4485_Capstone Frontend repository: https://github.com/marieliske/CS4485_Capstone_Frontend

2.2 Maintaining the "Single Source of Truth"

All project documentation was version-controlled in the team's GitHub repositories and updated via pull requests with peer review. The specification document was treated as a living artifact — any architectural change triggered a required update to both the spec and the relevant README sections before the PR could be merged.

Meeting Minutes (MOM #1 through MOM #9) were written after every team meeting and maintained in the MeetingMinutes folder of the repository, creating a timestamped record of all decisions, pivots, and action items. Major decisions documented this way included the pivot from webhook to GitHub Actions triggering, the SQLite to Supabase migration, the Supabase to Firebase migration, and the adoption of Workload Identity Federation.

Weekly individual status reports were maintained throughout the semester, with every team member logging hours, completed tasks, upcoming work, and blockers each week. This document served as the primary coordination artifact between meetings and gave the team continuous visibility into progress and blockers across all workstreams.

2.3 Key Milestones

Weeks 1–2 marked team formation, role assignments, GitHub and Jira setup, initial pseudocode for the weight-based mismatch detection algorithm, hash function design discussion, and API contract definition. This period established the foundational technical direction.

Week 3 was the most significant early milestone: the core backend pipeline was fully implemented. This included `ast_parser.py`, `fingerprint.py`, `comparator.py`, `alerts.py`, `flagging_threshold.py`, `report_generation.py`, and `run.py` — the complete analysis pipeline from source file to structured rot report. The system architecture document was also written this week.

Week 4 completed the project specification file, created the frontend GitHub repository, produced the initial Figma wireframes, and delivered the first working GitHub Actions integration with issue creation on detection.

Weeks 5–6 saw the frontend scaffolded with React pages, the initial database schema designed and implemented, a FastAPI bridge created for frontend-backend integration without a live database, and the first database migration from SQLite to PostgreSQL via Supabase.

Week 7 was the most significant infrastructure pivot of the project. A fundamental security flaw was identified in the Supabase architecture — the GitHub Action required direct database write access, which could not be secured without exposing credentials. The team made the decision to migrate entirely to Firebase and Firestore, implement OIDC-based authentication via Workload Identity Federation, and route all database writes through an authenticated Cloud Function. Firebase Authentication with GitHub OAuth was implemented, the frontend was fully connected to Firestore, and the first end-to-end pipeline was verified.

Week 8 completed the AI integration — Groq/Llama 3.3 70B suggestions were live in GitHub Issues and Firestore — and began the auto-fix feature development. Performance instrumentation was added to the frontend.

Week 9 delivered the complete auto-fix pipeline end-to-end and the npm setup wizard. Firestore query performance was substantially improved with parallel Promise.all calls, TTL caching, and pagination.

Week 10 was the major frontend milestone: a full UI redesign across all pages, incremental scanning with git diff, parallelized fingerprinting with ThreadPoolExecutor, and Firestore batch write chunking to handle large payloads.

Weeks 11–12 focused on bug fixes, final polish, demo preparation, and documentation finalization.

2.4 Team Roles & Leader Rotation

The team leader role rotated on a roughly bi-weekly basis. Each leader was responsible for the meeting agenda, MOM, and status report for their period.

The rotation was: Marie Liske (Weeks 1–2), Samuel Say (Weeks 3–4), Anusua Halder (Week 5), Portia Hamid (Weeks 6–7), Aaron Santhosh (Weeks 8–9), Suchi Jain (Weeks 10–11).

Technical responsibilities by member:

Anusua Halder served as the documentation and coordination lead throughout the project. She authored `flagging_threshold.py` and `report_generation.py` in the core backend pipeline. She wrote and maintained all Meeting Minutes (MOM #1 through #9) and the weekly status reports. She set up the initial `.github/workflows/docrot.yml` and `.docrot-config.json` in Week 4. She wrote Sections 5–7 of the project spec. On the frontend, she implemented Issues page UI polish and readability improvements including updated status labels, improved severity formatting, and a Resolve/Reopen button in the issue table. She updated and maintained both the frontend and backend READMEs throughout the semester and contributed to PR reviews and frontend testing.

Marie Liske served as a frontend lead and full stack developer. She created the frontend GitHub repository and initial React scaffold. She produced the initial Figma wireframes and wrote Sections 1–4 and 12–14 of the spec. In Week 3 she implemented config-driven documentation mapping and strengthened threshold handling and fingerprint baseline updating in the backend. In Week 7 she set up the Firebase Cloud Function backend with OIDC/Workload Identity Federation authentication, enabling the GitHub Action to write to Firestore securely. In Week 9 she created the npm setup wizard. She also helped lead scalability and GTM strategy research, fixed the GitHub OAuth login misconfiguration, and resolved merge conflicts across multiple PRs.

Suchi Jain served as a backend and frontend lead. She set up the main GitHub repository and granted TA access. She designed and implemented the initial database schema. She led the PostgreSQL/Supabase migration in Week 6. On the frontend, she built and fixed the rot gauge UI component, refactored the Issues page into an expandable row layout, reworked Scan History from split-view to row layout, unified UI patterns across pages, and added copy-to-clipboard for the repo setup command. She led GTM strategy research with Marie and led multiple rounds of frontend debugging and stabilization through Weeks 10–11.

Aaron Santhosh served as a frontend and backend integration lead. He created the initial Figma wireframes and implemented `comparator.py`, the core semantic diff engine. In Week 7, he connected the entire frontend to Firebase/Firestore, rewiring all major pages from mock data hooks to live Firestore queries, rebuilding the auth flow to GitHub-only, and implementing rot score calculation. He merged the AI integration branch and built the AI suggestions display in the scan history detail panel. He implemented and verified the full end-to-end auto-fix pipeline including PR creation, merge, and idempotency. He fixed the Firestore FieldPath dot-path bug

and the FlagReason priority bug, and implemented baseline auto-update on successful auto-fix PR merge. He also assisted with UI changes and suggestions along with other small bug fixes alongside other team members throughout the weeks.

Samuel Say served as the auth, deployment, and performance lead, while working on the frontend. He wrote `run.py`, the main pipeline entry point. He implemented Firebase Authentication with email/password and GitHub OAuth, deployed the frontend to Firebase Hosting, and added the GitHub Actions CI/CD workflow for auto-deploy. In Week 10 he implemented the full frontend UI redesign across all major pages, built the Tweaks Panel with persistent settings, added rot score visualizations, implemented incremental scanning with git diff, parallelized file fingerprinting with `ThreadPoolExecutor`, fixed the Firestore batch write limit with a chunked `commitBatches()` helper, and parallelized LLM suggestion calls. He also implemented optimistic UI updates, real-time search filtering, pagination, and a TTL cache for Firestore queries.

Portia Hamid served as the AI integration, security architecture, AST parser implementer, and frontend UI design lead. She led the initial pseudocode development for the weight-based mismatch detection algorithm and implemented `ast_parser.py` and `fingerprint.py` — the two foundational modules of the analysis pipeline. She authored `Architecture.md`. She identified the Supabase security flaw and co-architected the Firebase/OIDC/WIF migration with Marie. She completed the full AI integration including server-side Groq API calls via Cloud Function, API key management via Google Cloud Secret Manager, LLM output in GitHub Issues and Firestore, and support for OpenAI and Anthropic as alternative providers. In Week 10 she led the comprehensive frontend UI redesign, designing and implementing new layouts for the Dashboard, Scan History, Issues, Projects, Configuration, and User Settings pages. She implemented the environment-aware local preview mode, validated and merged the npm wizard PR, and provisioned Firebase/GCP admin access for teammates.

3. System Architecture & High-Level Design

3.1 System Overview

DocRot Detector is a cloud-native system organized around three primary layers: the analysis layer running as a GitHub Action, the persistence layer consisting of a Google Cloud Function and Firebase Firestore, and the presentation layer consisting of a React SPA with Firebase Authentication.

The analysis layer triggers on every push event. The workflow checks out the repository with full git history, authenticates to Google Cloud via OIDC and Workload Identity Federation, runs the DocRot scanner pipeline, and POSTs the scan payload to the Cloud Function using a short-lived Google-authenticated ID token.

The persistence layer's Cloud Function (ingestScan) validates the OIDC token and writes scan metadata, individual flags, AI suggestions, and fingerprint baselines into Firestore. The Firestore data model is:

repos/{repold} — repository metadata and latest scan pointer repos/{repold}/scan_runs/{scanId} — scan summary, status, and issue counts repos/{repold}/scan_runs/{scanId}/flags/{flagId} — individual documentation issue records
repos/{repold}/scan_runs/{scanId}/ai_suggestions/{suggestionId} — LLM-generated fix suggestions repos/{repold}/fingerprint_baselines/{branch} — serialized fingerprint baseline per branch

The presentation layer is a React SPA that authenticates users via Firebase GitHub OAuth and reads Firestore in real time, displaying repository health dashboards, per-repo rot scores, filterable issue tables, scan history, and AI suggestions.

Key Cloud Function endpoints:

- ingestScan (POST) — receives scan payload from GitHub Action, writes all data to Firestore
- applyFix (POST) — generates documentation patch and opens pull request against target repository
- closelssue (POST) — updates issue resolution status in Firestore

3.2 Architecture Diagrams

Data flow:

1. Developer pushes to a GitHub repository with DocRot configured
2. GitHub Actions workflow triggers on push event
3. Workflow authenticates to Google Cloud via OIDC / Workload Identity Federation, receiving a short-lived ID token
4. DocRot scanner runs the full pipeline: collects .py files (incremental via git diff when possible) → builds semantic fingerprints concurrently → compares against Firestore baseline → scores behavioral changes → maps to docs via .docrot-config.json → emits .docrot-report.json and .docrot-report.txt
5. If AI is enabled, flagged docs and change context are sent to Groq (Llama 3.3 70B) concurrently, receiving specific fix suggestions per flagged file
6. action_entrypoint.py POSTs the full scan payload to the Cloud Function using the OIDC ID token
7. Cloud Function validates token, chunks writes to stay under Firestore's 500-write limit, and persists all data
8. Frontend reads Firestore in real time and updates the dashboard automatically
9. On auto-fix request, the Cloud Function generates a patch and opens a pull request against the target repository

Firestore schema relationships:

- Each repo document links to its scan_runs subcollection
- Each scan_run links to flags and ai_suggestions subcollections
- Fingerprint baselines are stored per-branch to support multi-branch workflows

3.3 Key Architectural Decisions and Trade-offs

GitHub Actions over webhooks: Explored in Week 4, webhook-based triggering was found to require excessive repository permissions and a persistently hosted server. GitHub Actions eliminated both requirements and provided a cleaner security model through native OIDC token support.

Firebase over Supabase: After migrating from SQLite to Supabase in Week 6, the team identified that the GitHub Action required direct database write access, which necessitated exposing Supabase credentials as GitHub secrets — an inherent security flaw with no clean solution within the Supabase architecture. Firebase resolved this by allowing all writes to be routed through an authenticated Cloud Function, so the Action's only credential is a short-lived OIDC token.

OIDC / Workload Identity Federation over service account keys: Long-lived GCP service account keys stored as GitHub secrets represent a persistent credential leakage risk. Workload Identity Federation eliminates this by allowing GitHub Actions to authenticate using short-lived OIDC tokens that expire automatically, with no persistent secrets stored anywhere.

AST-based semantic fingerprinting over text diff: Text diffs generate false positives on cosmetic changes such as variable renames, comment edits, and code reformatting, and can miss semantic changes that look textually similar. Seven-dimensional semantic fingerprinting detects real behavioral drift while ignoring cosmetic noise, making the tool practical for active codebases.

Serverless Cloud Functions over a persistent backend server: Eliminates infrastructure management and scales automatically with usage. The trade-off is cold start latency on first invocation, which is acceptable given that scan ingestion is not a latency-critical operation.

Chunked Firestore batch writes: Firestore enforces a hard limit of 500 writes per batch. For large repositories with many flags and suggestions, a single batch.commit() call would silently fail. The chunked commitBatches() helper splits writes into groups of 400 to stay safely within the limit.

4. Implementation & AI Orchestration

4.1 Technologies Used

Python 3.10+ is the language of the core analysis pipeline, chosen for its native ast module and compatibility with the GitHub Actions runner environment.

The ast standard library module is used for all AST parsing and function extraction. No runtime execution of analyzed code is required, making the scanner safe for arbitrary third-party repositories.

GitHub Actions provides CI/CD automation. The DocRot workflow triggers on push events, runs the scanner, and sends results to the Cloud Function.

Google Cloud Functions (Node.js runtime) hosts the ingestScan, applyFix, and closeIssue endpoints. Serverless execution eliminates infrastructure management.

Firebase Firestore is the primary data store, holding all repository metadata, scan runs, flags, AI suggestions, and fingerprint baselines. Its document model maps naturally to the hierarchical scan/flag structure.

Firebase Authentication provides user authentication with GitHub OAuth as the sign-in provider.

Firebase Hosting serves the production frontend build.

React 19 is the frontend SPA framework. TypeScript provides type safety across all frontend code. Vite provides fast build tooling and the local development server.

Groq API (Llama 3.3 70B) powers the AI documentation fix suggestions. The API key is stored as a Google Cloud Secret and accessed server-side only, never exposed to the client.

Google Cloud Secret Manager manages the Groq API key securely on the server side.

Workload Identity Federation enables the GitHub Action to authenticate to Google Cloud using short-lived OIDC tokens without any stored service account keys.

The GitHub API is used for repository content access and pull request creation in the auto-fix workflow.

npm/npx distributes the setup wizard, allowing users to onboard any repository with a single command.

4.2 AI Build Strategy

The team used AI coding assistants including Claude, GitHub Copilot, and Cursor as pair programmers throughout development. The primary technique was structured prompting grounded in the project specification: each AI session began with the relevant spec section establishing architectural context, the module's interface contract defining inputs and outputs, and the specific task to be completed. This pattern consistently produced code that integrated cleanly with the existing codebase without requiring structural changes.

Claude Opus 4.6 was used to validate the initial pseudocode logic for the weight-based mismatch detection algorithm before implementation began. Claude was also used to generate the frontend UI redesign prototype that was subsequently implemented in code. The team

coordinated Claude usage across accounts during high-intensity periods to maximize AI-assisted iteration capacity.

The AI features within the product itself are governed by a separate prompt strategy. When the scanner flags documentation as potentially stale, the Cloud Function builds a prompt containing the flagged document's current content and a structured description of what changed in the code. This prompt is sent to Groq's Llama 3.3 70B, which returns specific, targeted edits — quoting existing text and providing corrected versions. The model is instructed to be concrete and actionable rather than general. OpenAI and Anthropic are supported as alternative providers via the `ai` block in `.docrot-config.json`, allowing teams to use their own API keys if preferred.

4.3 Human-in-the-Loop Interventions

Workload Identity Federation configuration required full manual intervention. AI assistants were unreliable for this specific setup due to frequently outdated and inconsistent documentation across Google Cloud and GitHub sources. The solution required identifying a working configuration from primary sources, manually configuring the OIDC provider and service account bindings in GCP, and documenting every step for the team.

Supabase security flaw identification was a human judgment call. The team identified that the architecture required the GitHub Action to have direct write access to the Supabase database, which could not be secured without exposing credentials. This assessment led directly to the Firebase migration decision, which was not surfaced by any automated tool or AI assistant.

Firestore FieldPath dot-path bug required manual diagnosis from production behavior. The `applyFix` Cloud Function was silently failing to update the fingerprint baseline after auto-fix PR merges because dots in Firestore key segments were being interpreted as path separators. The bug produced no errors and was only discoverable by observing that resolved flags reappeared on rescan.

AST edge cases for decorated and `async` functions were not handled correctly by the initial fingerprinter. The visitor failed silently on files using these patterns, missing findings entirely. These cases were identified through targeted test fixtures and manually patched.

Firestore batch write limit caused silent failures on large scan payloads. The hard limit of 500 writes per batch was not surfaced as an error — the batch simply committed fewer records than expected. Manual diagnosis from Firestore data inspection identified the cause, leading to the chunked `commitBatches()` implementation.

Threshold calibration was not well-handled by AI-generated defaults. The initial thresholds treated all change types uniformly, generating alert fatigue on minor refactors. The team manually tuned thresholds based on observed alert patterns across test repositories to reach a configuration that flagged meaningful drift without overwhelming developers.

4.4 Codebase

Backend repository: https://github.com/SuchiiJain/CS4485_Capstone

Frontend repository: https://github.com/marieliske/CS4485_Capstone_Frontend

Live dashboard: <https://docrot-detector.web.app/>

GTM Strategy Document:

<https://hulking-blouse-367.notion.site/DocRot-Detector-GTM-Strategy-3447bf3fb7b880a1bd32fc67570bb4e9>

Core backend modules:

- `action_entrypoint.py` — action runtime orchestration, GitHub issue handling, and backend POST
- `src/run.py` — main scan pipeline with incremental git diff scanning and parallelized fingerprinting via `ThreadPoolExecutor`
- `src/ast_parser.py` — AST parsing, function/method extraction, parent-class annotation, stable function ID generation, and public/private visibility detection
- `src/fingerprint.py` — seven-dimensional semantic feature extraction per function
- `src/comparator.py` — fingerprint comparison and behavioral change scoring
- `src/alerts.py` — documentation mapping and threshold alert evaluation
- `src/ai_suggestions.py` — parallelized LLM-powered documentation fix suggestions
- `src/report_generation.py` — `.docrot-report.json` and `.docrot-report.txt` generation
- `src/persistence.py` — fingerprint baseline persistence
- `functions/index.js` — Cloud Function ingest endpoint with chunked Firestore batch writes

5. System Performance & Validation

5.1 Testing and Validation

Backend validation was performed using `pytest` with test fixtures covering both well-documented Python files (expected: zero findings) and intentionally rotted files with known mismatches (expected: specific finding types and counts). A dedicated test repository ("docrot-demo") was created with intentional documentation rot across multiple functions to enable realistic end-to-end pipeline testing. The full pipeline was verified repeatedly across the test repository: push to GitHub, Action triggers, scanner runs, payload POSTs to Cloud Function, Firestore writes, frontend updates.

API endpoint testing was performed using manual inspection of Cloud Function logs and Firestore data to verify correct payload structure, authentication behavior, and write semantics.

Edge cases tested included missing OIDC tokens, malformed payloads, and repositories with no Python files.

Frontend validation was manual and end-to-end, covering all major user flows: GitHub OAuth login and logout, repository onboarding via the npm wizard, scan history viewing and filtering, issue table filtering and detail panel expansion, AI suggestion display, auto-fix trigger and PR verification, and configuration view rendering. Samuel instrumented Firestore queries with a performance timing utility to surface per-query latency and identify bottlenecks, which led to the parallelization and caching improvements in Weeks 9–10.

5.2 Technical Performance Metrics

Incremental scanning with git diff reduced the set of files analyzed on typical pushes from the full repository to only the changed files, substantially reducing scan time for active repositories with large codebases.

Parallelized fingerprinting with ThreadPoolExecutor eliminated sequential file processing, with all changed files now parsed concurrently. For a repository with 20 changed files, this reduced fingerprinting time proportionally to the slowest single file rather than the sum of all files.

Firestore query optimization in the frontend reduced page load times for the Projects and Issues pages by replacing sequential Firestore queries with parallel Promise.all calls, adding 25-row pagination to limit rendered records, and implementing a 60-second TTL in-memory cache for repeated navigations. The closeIssue function was rewritten to use a direct Firestore path lookup rather than scanning all repos, reducing reads from $O(n \text{ repositories})$ to a single write.

Firestore batch write chunking resolved silent failures on large scan payloads by splitting writes into groups of 400 to stay under Firestore's hard limit of 500 writes per batch.

5.3 AI Model Performance

The AI suggestion feature uses Groq's Llama 3.3 70B to generate documentation fix suggestions for flagged files. Suggestions are evaluated qualitatively — the model is prompted to produce specific, actionable edits quoting existing text and providing corrected versions, rather than general guidance.

Parallelizing LLM calls across all flagged documents in a single scan reduced total AI suggestion generation time from sequential (sum of all calls) to approximately the latency of the slowest single call, enabling the feature to remain practical even for scans with multiple flagged documents.

The core detection engine is deterministic and AST-based, so traditional accuracy metrics apply to the scanner itself rather than the LLM layer. Detection quality was validated against the intentionally rotted test repository, confirming that all seeded rot instances were correctly flagged and that clean files produced zero findings.

6. Lessons Learned

6.1 Insights Gained

The most significant shift this project required was moving from individual coding toward AI-orchestrated, collaborative engineering at a scale where the quality of the specification directly determines the quality of the output. Working effectively with AI assistants is a distinct skill: a vague prompt grounded in no context produces generic code that requires heavy modification, while a prompt derived from a precise spec section produces code that integrates immediately. The team learned this early and the investment in maintaining a detailed, current spec paid dividends every week.

The project also reinforced that documentation is not a downstream artifact — it is an upstream engineering input. The system the team built exists precisely because documentation rot is costly, and the team experienced this firsthand. Decisions that were made clearly in a meeting but not captured in the MOM or spec required repeated re-explanation in subsequent weeks. Decisions that were documented cleanly stayed decided.

The Firebase migration in Week 7 was the clearest example of security as an architectural concern rather than a feature to add later. The Supabase architecture was functional, but it had a structural security flaw baked into its design. Recognizing and acting on that flaw mid-semester, even at the cost of a full week of migration work, was the right call and resulted in a significantly more secure and maintainable system.

6.2 Lessons from Challenges

Workload Identity Federation setup: GCP and GitHub documentation for this integration was frequently outdated or contradictory across sources. AI assistants consistently produced incorrect configurations. The solution was to treat it as a research problem, identify the most recent working primary source, follow it exactly, and document every step in the team's knowledge base for future reference.

Supabase to Firebase migration: The decision to migrate mid-semester required the entire team to rapidly context-switch to a new platform. The lesson was that security architecture decisions made early in a project are expensive to change later, and that evaluating security properties — not just functional ones — should be part of the initial technology selection process.

Firestore silent failures: Both the batch write limit and the FieldPath dot-path bug failed silently, producing no errors and incorrect behavior. The lesson was that cloud database operations

require active verification of writes, not just absence of errors, particularly when the data model involves nested paths or large batch operations.

AST edge cases: The initial implementation was validated against simple Python files and missed decorated and async function patterns that are common in real codebases. The lesson was that test fixtures need to reflect the full diversity of real-world code patterns, not just the common cases.

AI tool limits during high-intensity periods: Several team members hit Claude usage limits during peak development weeks. The team learned to plan AI-intensive work across the week rather than concentrating it, and to distribute usage across accounts when necessary.

6.3 Skills Developed

The team developed significant skills in AI-augmented software engineering, including structured prompting, prompt library maintenance, critical evaluation of AI-generated code, and understanding the boundary between what AI handles well and where human judgment is required.

Cloud architecture skills grew substantially across the team, including serverless design patterns with Google Cloud Functions, Firestore data modeling for hierarchical scan data, Firebase Authentication integration, and production deployment workflows.

CI/CD security skills were developed through the implementation of Workload Identity Federation, least-privilege service account design, and OIDC-based keyless authentication — practices that are increasingly standard in production engineering environments.

Frontend engineering skills grew across the team through work on React 19, TypeScript, Vite, Firestore client SDK integration, performance optimization patterns, and production-grade UI design across a multi-page application.

Collaborative engineering practices were reinforced throughout the semester including branch-and-PR workflows, peer code review, documentation as a first-class engineering artifact, and async coordination through weekly status reports and meeting minutes.

6.4 Individual Reflections

Anusua Halder: This capstone was my first experience contributing to a real multi-person software project with a live deployment. My primary focus was documentation and coordination, which taught me that these contributions are load-bearing, a team without clear meeting minutes and an up-to-date spec loses hours to repeated context-setting every week. I also contributed `flagging_threshold.py` and `report_generation.py` early in the semester, which gave me hands-on experience with the backend pipeline and helped me understand the system well enough to keep the documentation accurate as it evolved. On the frontend I worked on the Issues page polish and the Resolve/Reopen feature, which was my first experience making code contributions to a React codebase. Learning Git and GitHub workflows from scratch was

my biggest personal challenge this semester - by the end, branch-and-PR workflows felt natural and I was making README contributions and opening PRs independently.

Marie Liske: Leading the frontend and infrastructure work as well as working on backend gave me a full stack end-to-end experience at a scale and complexity beyond any previous project. The most valuable technical lesson was the importance of security-first architecture decisions. The Supabase to Firebase migration was expensive and cost a lot of time, but the decision was correct. Setting up the Firebase Cloud Function with OIDC and Workload Identity Federation was one of the hardest technical problems I worked on this semester, and getting it working was one of the most satisfying. Building the npm wizard was a reminder that developer experience is a product feature, making onboarding take one command instead of ten steps meaningfully changes whether people actually use a tool.

Suchi Jain: Working on both the database architecture and the frontend gave me an appreciation for how much the data model affects everything downstream. The SQLite to Supabase to Firebase migrations were painful, but each one taught me something new about the trade-offs between different database and hosting architectures. The frontend work in Weeks 10–11 — the rot gauge, the refactored issues and scan history layouts, the UI unification — taught me how much polish matters for a tool that people actually need to look at and use every day.

Aaron Santhosh: Connecting the entire frontend to Firestore in Week 7 was the most intense single-week contribution I made this semester, and it gave me a deep understanding of how a real-time database-backed frontend works end-to-end. Implementing the auto-fix pipeline was the most satisfying feature I built. Taking a system that detected a problem and making it capable of fixing it on its own closed a loop that made the whole tool feel real. The Firestore silent failure bugs were frustrating precisely because they required reasoning about what didn't happen, which is a different debugging skill than tracking down an error message. Being able to work in a capstone group like this on a difficult project concept was some of the most satisfying work I've done while at UTD and it's both a memory and an experience I will look back on fondly.

Samuel Say: The Week 10 work — UI redesign, incremental scanning, parallelized fingerprinting, Firestore batch chunking, parallelized LLM calls — compressed more engineering work into a single week than I had done before in this project. The performance work in particular taught me that optimization requires measurement: without the instrumentation added in Week 8, the Week 9–10 improvements would have been guesswork. The Firestore batch write bug was a good example of why distributed systems require a different debugging mindset — the system appeared to work, but the data was wrong in ways that only became visible when looking at Firestore directly. Additionally, working on frontend as someone with no previous experience doing frontend work was a great learning experience. Even though AI was used heavily, guiding it to function properly still taught me a lot.

Portia Hamid: I built most of the backend, including the system architecture, the AST parser, the fingerprinting engine, the GitHub Actions integration, and the AI suggestion layer.

Implementing `ast_parser.py` and `fingerprint.py` at the start meant I had to understand the full detection pipeline before writing a single line of application code, and that foundation made every later contribution easier. The biggest lesson was that owning a system end-to-end means owning its weaknesses too. Recognizing that the Supabase architecture had a structural security flaw and pushing to migrate to Firebase mid-semester taught me that good architectural judgment sometimes means accepting a week of expensive rework rather than shipping something you know is wrong. The AI integration was the feature I cared most about, and the constraint I held onto, that it had to work with zero setup out of the box, taught me how much developer experience considerations should drive technical choices rather than being layered on at the end. The Week 10 frontend redesign stretched me in a different direction, where I drove the design language by iterating with Claude Design and handed the resulting code to Samuel for implementation.

7. Conclusion & Future Work

7.1 Summary of Key Accomplishments

DocRot Detector successfully met and exceeded its original objectives for the Spring 2026 semester. The team delivered a fully deployed, end-to-end documentation rot detection platform that operates automatically on every push, requires minimal setup from repository owners, and surfaces findings through a polished, live web dashboard.

Core technical accomplishments include a complete AST-based semantic fingerprinting and comparison pipeline, a secure serverless architecture with no long-lived credentials anywhere in the CI/CD pipeline, a Groq-powered AI suggestion layer that generates actionable documentation fix proposals, an auto-fix feature that closes the loop from detection to resolution via automated pull requests, an npm setup wizard enabling single-command onboarding, and a fully featured React dashboard with real-time Firestore integration.

The project demonstrated that a six-person team, using AI-assisted engineering practices grounded in a well-maintained specification, can deliver a non-trivial cloud-native application over a single academic semester. DocRot Detector is a functional tool that addresses a real problem in software engineering practice and is deployed and accessible at <https://docrot-detector.web.app/>.

7.2 Future Enhancements

The current release is a strong MVP with clear paths for expansion.

Multi-language support is the highest priority for post-MVP development. The detection pipeline was designed with language extensibility in mind — extending it to support JavaScript/TypeScript JSDoc and Java Javadoc would expand the addressable market substantially.

LLM-augmented semantic checking would add a layer beyond AST analysis, detecting semantic drift between docstring descriptions and function logic that structural analysis cannot catch. This would complement rather than replace the existing fingerprinting approach.

A VS Code extension would surface rot findings inline as the developer writes code, shortening the feedback loop from batch scan on push to real-time detection in the editor.

GitHub App and pull request integration would enable DocRot to comment findings directly on pull requests and optionally block merges when high-severity rot is introduced, integrating detection into the code review workflow.

Historical trend analysis would track documentation health over time per repository, surfacing rot velocity metrics — how fast documentation quality is degrading — alongside per-scan findings.

A team dashboard with multi-user views and aggregated rot metrics across a portfolio of repositories would make the tool practical for engineering managers and platform teams tracking documentation health organization-wide.

References

[1] Python Software Foundation. ast — Abstract Syntax Trees. Python 3.11 Standard Library Documentation. <https://docs.python.org/3/library/ast.html>

[2] Google Cloud. Workload Identity Federation. <https://cloud.google.com/iam/docs/workload-identity-federation>

[3] Google Cloud. Cloud Functions Documentation. <https://cloud.google.com/functions/docs>

[4] Firebase. Firebase Authentication Documentation. <https://firebase.google.com/docs/auth>

[5] Firebase. Cloud Firestore Documentation. <https://firebase.google.com/docs/firestore>

[6] GitHub. GitHub Actions Documentation. <https://docs.github.com/en/actions>

[7] Groq. Groq API Documentation. <https://console.groq.com/docs>

[8] React. React 19 Documentation. <https://react.dev>

[9] Microsoft. TypeScript Documentation. <https://www.typescriptlang.org/docs/>

[10] Vite. Vite Documentation. <https://vitejs.dev>

[11] PEP 257 – Docstring Conventions. <https://peps.python.org/pep-0257/>

Acknowledgements

The team thanks Professor Sridhar Alagar for his guidance throughout CS 4485 and for encouraging the team to pursue a project that addresses a real gap in software engineering practice. We thank Project Coordinator Adarsh Gella for keeping the course structure organized and for his feedback on our weekly status reports and deliverables.

We also thank the open-source and developer tool communities whose work made this project possible, including the Python core team, the Firebase and Google Cloud teams, the Groq team, and the React and Vite communities. Finally, we thank each other — this project required sustained effort and genuine collaboration across six people with different skill sets and responsibilities, and every member contributed meaningfully to the final result.